



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de **HONORIS UNITED UNIVERSITIES**



École Marocaine des Sciences de l'Ingénieur de Tanger

Département Ingénierie Informatique et Réseaux

MINI-PROJET PYTHON

Programmation Python et Frameworks

PORTAL

Application de Gestion des Présences aux Séances de Cours

Réalisé par :

BAKKALI Owais

AHRRAR Amjad

HABZ Amine

DOUKHANA Bachar

MESBAHI Bilal

Encadrante : Pr. Abroun Soundouss

Membres du jury : Pr. Abroun Soundouss

Remerciements

Nous tenons à exprimer notre profonde gratitude envers notre encadrante, **Pr. Abroun Soundouss**, pour son accompagnement bienveillant, sa disponibilité remarquable et les précieux conseils qu'elle nous a prodigués tout au long de la réalisation de ce mini-projet. Ses orientations techniques et méthodologiques ont été déterminantes pour la qualité du travail accompli.

Nous remercions chaleureusement l'École Marocaine des Sciences de l'Ingénieur de Tanger pour l'excellente formation d'ingénieur qu'elle nous dispense, ainsi que pour les ressources pédagogiques mises à notre disposition. Ce cadre stimulant nous a permis de relever ce défi avec sérieux et enthousiasme.

Nos remerciements s'adressent également à l'ensemble des enseignants du département d'Informatique, dont les cours en Python, bases de données et génie logiciel nous ont fourni les compétences nécessaires à la réalisation de ce projet. Enfin, nous remercions nos familles et amis pour leur soutien moral et leur patience tout au long de ce travail.

Table des matières

Remerciements	i
1. Description du système	1
2. Description des acteurs	1
3. Structure des données	2
4. Description des fonctions principales	3
5. Stack technique et dépendances	6
6. Captures d'écran	6
Conclusion	8

1. Description du système

Le projet **PORTAL** (v2.4.0) est une application web complète de **gestion des présences aux séances de cours** développée en Python avec le framework **Django 6.0.4** pour le back-end et **React 19** (via **Vite 8**) pour le front-end. Destinée à un établissement d'enseignement supérieur tel qu'EMSI, elle offre à trois catégories d'utilisateurs — administrateur, enseignant, étudiant — des interfaces dédiées adaptées à leurs responsabilités.

Les données sont persistées dans une base **SQLite** locale via l'ORM Django. L'architecture est de type *découplée* : l'interface React communique exclusivement avec le back-end Django via une **API REST** exposée par Django REST Framework 3.17. Cette séparation garantit modularité, testabilité et maintenabilité. La communication inter-domaines est gérée par **django-cors-headers**.

Les grandes fonctionnalités couvertes sont :

- Authentification par email/mot de passe avec redirection automatique vers le tableau de bord du rôle concerné ; modification du mot de passe et upload de photo de profil depuis la barre supérieure.
- Gestion complète des utilisateurs (étudiants, enseignants, administrateurs) : génération automatique d'email (domaine selon le rôle), de matricule et du profil associé.
- Affectation de modules et de classes aux enseignants via un catalogue hiérarchique couvrant l'ensemble des filières EMSI.
- Gestion du catalogue de cours (**Course**) et création de séances (**Session**) liées à un cours, une classe et un enseignant.
- Saisie et consultation des présences par séance avec sauvegarde groupée (*bulk_create* — upsert atomique).
- Saisie groupée des notes par classe (*bulk_submit_grades*) avec recalcul automatique de la moyenne générale du bulletin.
- Suivi des bulletins de notes (**ReportCard + Grade**) par année académique et semestre, avec vue « Relevé global ».
- Gestion des absences (**Absence**) et workflow de justification : soumission par l'étudiant, validation ou rejet par l'administrateur.
- Demande et suivi de documents administratifs (**DocumentRequest**) : attestation de scolarité ou de réussite.
- Système de notifications ciblées (**Notification**) avec liste de destinataires ManyToMany et affichage du rôle de l'expéditeur.
- Calendrier d'événements académiques (**CalendarEvent**) ciblés par filière ou globaux, éditables par enseignants et administrateurs.
- Emploi du temps hebdomadaire par classe (**ClassSchedule**, JSONField 5 jours), créé et mis à jour par l'administrateur, consulté par enseignants et étudiants.
- Thème clair/sombre (Glassmorphism) commutable à tout moment ; persistance du choix dans le `localStorage`.

2. Description des acteurs

Acteur	Description et responsabilités
Administrateur	Contrôle total de la plateforme : création, modification et suppression des comptes (étudiants, enseignants, administrateurs) avec dérivation automatique de la classe selon la filière et l'année ; affectation de modules et de classes aux enseignants depuis un catalogue hiérarchique complet ; gestion du catalogue de cours ; publication et édition de l'emploi du temps hebdomadaire par classe (ajout/suppression de créneaux, sélection de l'enseignant) ; gestion globale du calendrier académique (ajout, édition, suppression d'événements tous enseignants confondus) ; validation ou rejet des justifications d'absence et des demandes de documents ; diffusion d'annonces globales à tous les étudiants.
Enseignant	Consulte les séances qui lui sont attribuées et l'emploi du temps des classes dont il est responsable ; crée des séances de cours et saisit les présences/absences étudiant par étudiant puis les enregistre en masse ; saisit les notes par module, semestre et type d'évaluation (CC, Examen final) pour une classe entière ; envoie des notifications à tous les étudiants ; ajoute et gère ses propres événements dans le calendrier.
Étudiant	Consulte son emploi du temps hebdomadaire, ses enregistrements de présence (par séance avec cours, classe et date), ses bulletins de notes par année et semestre (avec vue « Relevé global ») et ses notifications (avec suppression individuelle ou globale) ; soumet une justification textuelle pour chaque absence non encore traitée ; effectue des demandes d'attestation administrative ; modifie son mot de passe et sa photo de profil.

3. Structure des données

Les données sont organisées en modèles Django persistés automatiquement dans SQLite via les migrations. Chaque entité possède un identifiant primaire auto-incrémenté (`BigAutoField`).

Modèle	Champs principaux
CustomUser	id, username, email, password (hashé Django), role (<i>student</i> / <i>teacher</i> / <i>admin</i>), matricule (unique), gender (M/F), profile_picture (ImageField, media/profile_pics/)
StudentProfile	user (OneToOne), filiere, annee_etude, numero_etudiant, tutor_name
TeacherProfile	user (OneToOne), departement, matiere (TextField/JSON), classes (TextField/-JSON)
AdminProfile	user (OneToOne), service
Course	id, name, code, created_by (FK User)
Session	id, course (FK), teacher (FK, <i>limit_choices_to role=teacher</i>), target_class, date, start_time, end_time, created_by (FK)
Attendance	id, session (FK), student (FK), present (BooleanField) — contrainte unique_together (session, student)
ReportCard	id, student (FK), academic_year, semester, general_average (FloatField)
Grade	id, report_card (FK), subject, evaluation_type, value (FloatField), is_rattrapage (BooleanField)
Absence	id, student (FK), teacher (FK, nullable), subject, date_seance, is_present, justification_text, justification_status (Pending/Validated/Rejected)
ClassSchedule	id, target_class (unique), schedule_data (JSONField : liste de 5 listes, une par jour ouvré, chaque entrée : {time, name})
Notification	id, title, content, date_envoi (auto), type_notif, sender (FK), recipients (ManyToMany)
CalendarEvent	id, title, description, start_time, end_time, event_type (Cours/TD/Examen), created_by (FK), professor (FK nullable, teacher only), target_classes (ex. "1A_IIR, 2A_IIR" ou "All Classes")
DocumentRequest	id, student (FK), document_type (Scolarité/Reussite), status (Pending/Validated/Rejected), created_at (auto)

TABLE 2 – Modèles de données du projet PORTAL (version finale)

Relations clés : **Attendance** relie **Session** et **CustomUser** (étudiant) via une contrainte d'unicité garantissant l'absence de doublons ; **Grade** est systématiquement rattaché à un **ReportCard** identifié par le triplet (étudiant, année, semestre) ; **ClassSchedule** est un modèle singleton par classe (contrainte unique sur **target_class**).

4. Description des fonctions principales

Application accounts — Gestion des utilisateurs (accounts/views.py)

- **login(email, password)** : recherche du **CustomUser** par email, authentification Django, renvoi du sérialiseur complet (incluant les profils imbriqués **student_profile**, **teacher_profile**, **admin_profile**).
- **create_user()** : génération automatique de l'email (domaine selon le rôle), d'un **username** unique et d'un **matricule** préfixé ; création du profil approprié ; pour les étudiants, dérivation de la classe (**filiere**) à partir du secteur, de l'année et de la spécialité.
- **change_password(email, old_password, new_password)** : vérification de l'ancien mot de passe avant mise à jour.
- **upload_profile_picture(email, profile_picture)** : réception multipart, stockage dans **media/profile_p**, renvoi du sérialiseur mis à jour.
- **assign_modules(teacher_id, modules, classes)** : affecte des listes sérialisées JSON de modules et de classes à un **TeacherProfile** ; les enseignants ne voient que leurs modules et classes dans leurs tableaux de bord.
- **demo_users()** : renvoie un exemple de chaque rôle (utilisé en développement).

Application portal — Gestion académique (portal/views.py)

- **CalendarEventViewSet** : filtrage intelligent selon le rôle — admin voit tout, enseignant voit ses événements, étudiant voit les événements globaux et ceux ciblant sa filière (via **target_classes**).

- `CourseViewSet` : CRUD du catalogue de cours (nom, code, créateur).
- `SessionViewSet` : création de séances ; filtrage par enseignant (`?teacher=id`), par classe (`?target_class=`) ou par cours (`?course=`).
- `AttendanceViewSet.bulk_create(records)` : upsert atomique de toutes les présences d'une séance en un seul appel POST via `update_or_create` sur (`session_id`, `student_id`) ; filtrage disponible par session, étudiant ou enseignant.
- `GradeViewSet.bulk_submit_grades()` : saisie groupée des notes d'une classe entière ; création ou mise à jour du `ReportCard` correspondant au triplet (étudiant, année, semestre) ; recalcul automatique de `general_average`.
- `AbsenceViewSet` : gestion du workflow justification (PATCH sur `justification_status`).
- `ClassScheduleViewSet` : GET filtré par `target_class` ; POST (création) ou PATCH (mise à jour) du `JSONField` de 5 jours.
- `NotificationViewSet` : création avec liste de destinataires (ManyToMany) ; les sérialiseurs enrichissent la réponse avec `sender_name` et `sender_role`.
- `DocumentRequestViewSet` : dépôt et suivi des demandes d'attestations.

Sérialiseurs notables (`portal/serializers.py`)

- `SessionSerializer` : expose `course_name`, `teacher_name` et `created_by_name` en champs calculés (`SerializerMethodField`).
- `AttendanceSerializer` : imbrique `session_details` (`SessionSerializer`) et `student_details` (`UserSerializer`) pour un accès complet depuis le front-end.
- `ReportCardSerializer` : imbrique la liste `grades` (`GradeSerializer`, read-only) et `student_details`.

Interface utilisateur React (`frontend/src/`)

- `App.jsx` : gestion de l'état d'authentification (`localStorage`), du thème clair/sombre (attribut `data-theme` sur `<html>`) et de l'onglet actif ; rendu conditionnel du dashboard selon `user.role`.
- `Login.jsx` : formulaire email/mot de passe avec gestion des erreurs serveur et bascule de thème indépendante.
- `AdminDashboard.jsx` : annuaire complet, création d'utilisateurs avec dérivation de classe, affectation de modules/classes via catalogue hiérarchique EMSI, gestion du calendrier global (CRUD avec sélection du professeur), emploi du temps par classe (ajout/suppression de créneaux), notifications globales, file de validations (absences + documents), gestion du catalogue de cours.
- `TeacherDashboard.jsx` : emploi du temps de ses classes (lecture seule), liste des événements (CRUD), création de séances, saisie des présences session par session (chargement des enregistrements existants à la sélection), saisie des notes par classe avec filtres module/semestre/type, gestion des moyennes des étudiants tutorés, diffusion de notifications.
- `StudentDashboard.jsx` : emploi du temps hebdomadaire (5 colonnes colorées), liste des événements à venir, notifications avec suppression unitaire et globale, bulletins de notes avec onglets par année et semestre (vue « Relevé global » multi-années), enregistrements de présence (tableau avec cours/classe/date/statut), gestion des absences et formulaire de justification, demandes de documents.
- `TopBar.jsx` : modal de paramètres (changement de mot de passe + upload de photo de profil via `FormData`).
- `Sidebar.jsx` : navigation contextuelle selon le rôle ; affichage de l'avatar (photo ou initiales), du nom et du rôle ; pied avec crédits de l'équipe et version.
- `api.js` : instance Axios centralisée pointant vers `http://127.0.0.1:8000/api/`.

Scripts utilitaires (`backend/`)

- `populate_db.py` : réinitialise la base et insère des utilisateurs démo (étudiant *Nizar EL IDRYSY* lié à l'enseignante *Hajar CHABLI* comme tutrice, administrateur, second enseignant), des événements, des notifications, des bulletins multi-annuels (2024-2025 S1+S2, 2025-2026 S1) et des absences.

- `seed_grades.py` : génère aléatoirement des bulletins pour les années passées (2023-2024, 2024-2025) pour tous les étudiants existants, avec des notes entre 6 et 20 et des flags `is_rattrapage`.
- `test_auth.py` : script de vérification des mots de passe des utilisateurs existants.

Exemple de saisie groupée des présences (back-end Django) :

```

@action(detail=False, methods=['post'])
def bulk_create(self, request):
    records = request.data.get('records', [])
    updated_count = 0
    for record in records:
        session_id = record.get('session_id')
        student_id = record.get('student_id')
        present = record.get('present', False)
        if session_id is None or student_id is None:
            continue
        Attendance.objects.update_or_create(
            session_id=session_id,
            student_id=student_id,
            defaults={'present': present}
        )
        updated_count += 1
    return Response(
        {'message': f'Successfully recorded {updated_count} attendance records'},
        status=status.HTTP_200_OK
    )

```

Listing 1 – Action `bulk_create` du ViewSet Attendance (`portal/views.py`)

Exemple de saisie groupée des notes avec recalcul de la moyenne :

```

@action(detail=False, methods=['post'])
def bulk_submit_grades(self, request):
    subject = request.data.get('subject')
    evaluation_type = request.data.get('evaluation_type', 'Examen')
    grades_data = request.data.get('grades', [])
    academic_year = request.data.get('academic_year', '2023-2024')
    semester = request.data.get('semester', 'S1')

    for grade_entry in grades_data:
        rc, _ = ReportCard.objects.get_or_create(
            student_id=grade_entry['student_id'],
            academic_year=academic_year, semester=semester
        )
        Grade.objects.update_or_create(
            report_card=rc, subject=subject,
            evaluation_type=evaluation_type,
            defaults={'value': float(grade_entry['value'])}
        )
    all_grades = Grade.objects.filter(report_card=rc)
    rc.general_average = round(
        sum(g.value for g in all_grades) / all_grades.count(), 2
    )
    rc.save()
    return Response({'message': f'Successfully updated grades'},
        status=status.HTTP_200_OK)

```

Listing 2 – Action `bulk_submit_grades` du ViewSet Grade (`portal/views.py`)

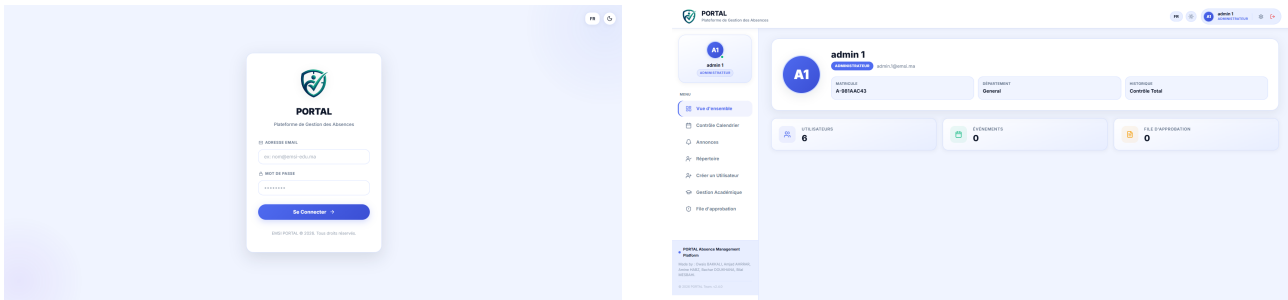
5. Stack technique et dépendances

Couche	Technologie	Version / Rôle
Back-end	Django	6.0.4 — framework principal
	Django REST Framework	3.17.1 — API REST
	django-cors-headers	4.9.0 — autorisation CORS
	Pillow	12.2.0 — traitement images (photos de profil)
	python-dotenv	1.2.2 — variables d'environnement
	SQLite	intégré Django — base de données
Front-end	React	19.2.5 — interface utilisateur
	Vite	8.0.10 — bundler / HMR
	Axios	1.15.2 — requêtes HTTP
	Lucide React	1.11.0 — icônes SVG
	React Router DOM	7.14.2 — routage SPA
	i18next	26.0.9 — internationalisation

TABLE 3 – Stack technique du projet PORTAL

6. Captures d'écran

Remarque : les noms de fichiers ci-dessous correspondent aux captures à insérer dans le répertoire du projet Overleaf. Remplacer les fichiers **.png** par les véritables captures de l'application.



(a) Page de connexion

(b) Tableau de bord Administrateur

FIGURE 1 – Authentification et accueil

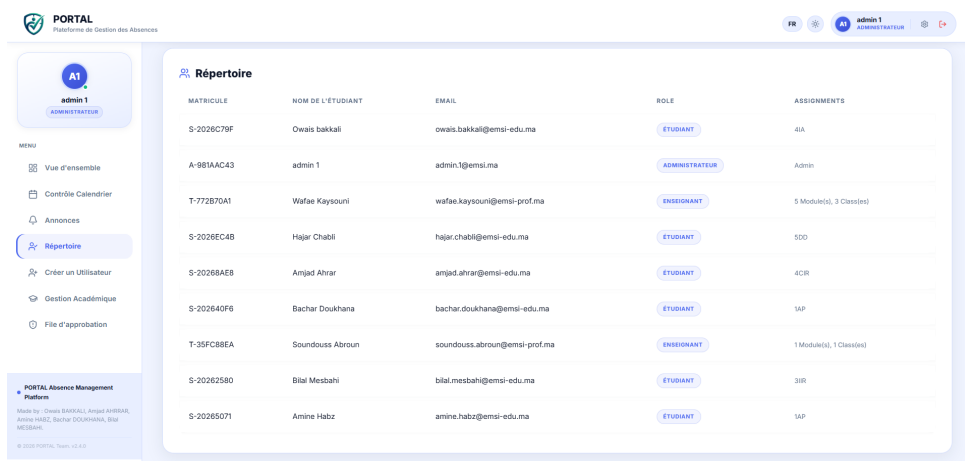
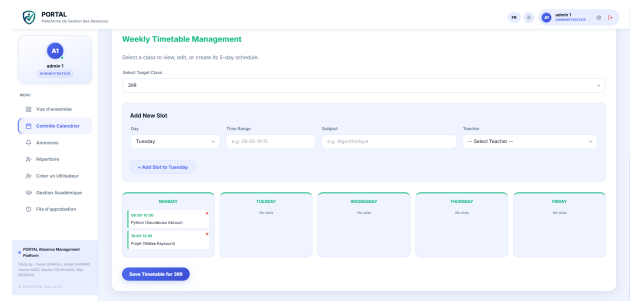
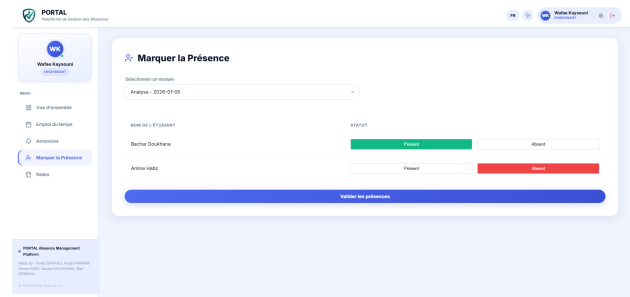


FIGURE 2 – Annuaire des utilisateurs — création avec dérivation automatique de classe



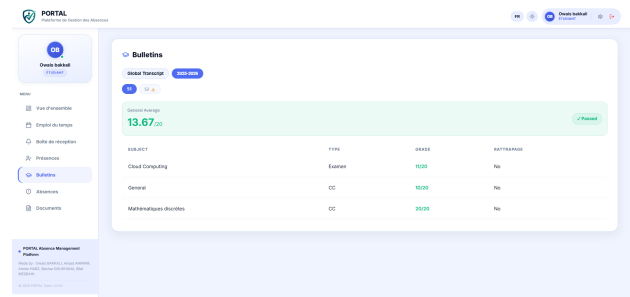
(b) Éditeur d'emploi du temps par classe (Admin)

FIGURE 3 – Administration académique



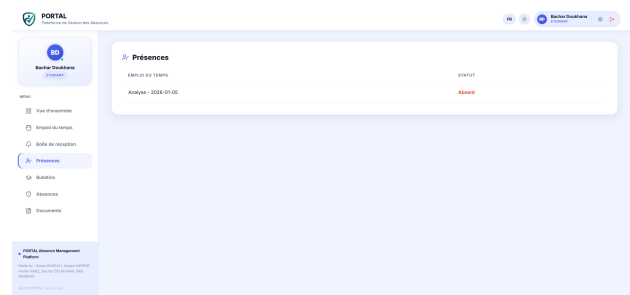
(b) Saisie des présences par session

FIGURE 4 – Gestion des séances et des présences



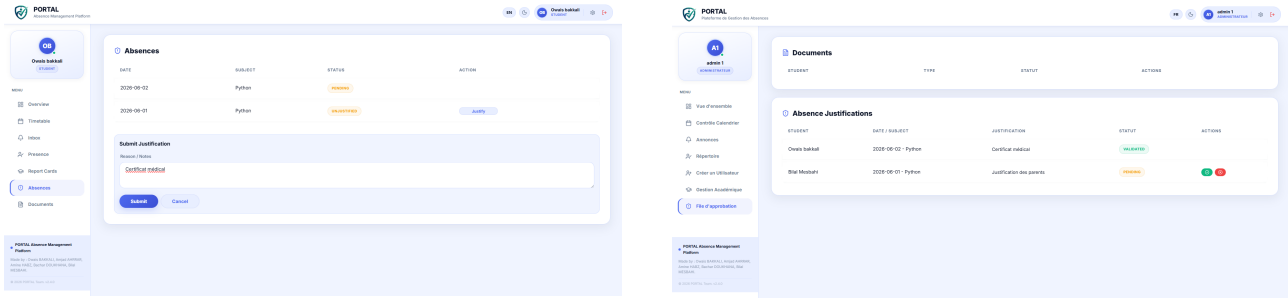
(b) Relevé global multi-années (Étudiant)

FIGURE 5 – Gestion des notes et bulletins



(b) Enregistrements de présence (Étudiant)

FIGURE 6 – Emploi du temps et suivi des présences



(a) Justification d'absence (Étudiant)

(b) File de validation (Administrateur)

FIGURE 7 – Gestion des absences et validations administratives

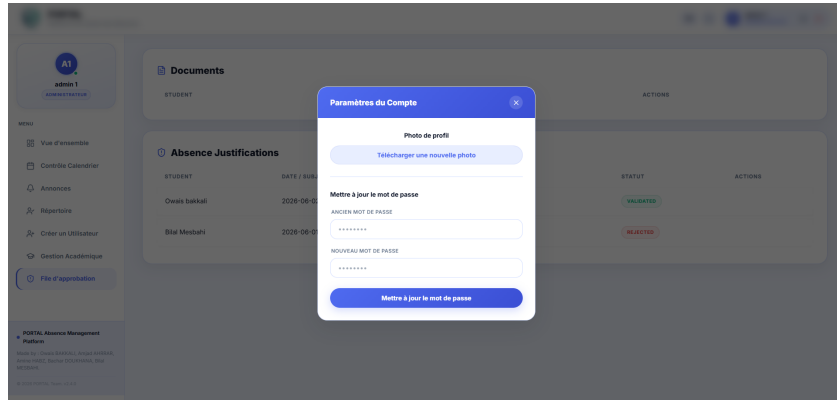


FIGURE 8 – Modal de paramètres — changement de mot de passe et photo de profil

Conclusion

Le projet **PORTAL** répond intégralement aux spécifications du mini-projet Python tout en allant au-delà par l'étendue de ses fonctionnalités. L'application est fonctionnelle, persistante et modulaire. Elle illustre concrètement la mise en œuvre de concepts avancés : architecture découplée REST, modélisation relationnelle complexe (contraintes d'unicité, relations ManyToMany, JSONField), sérialiseurs imbriqués, actions personnalisées sur les ViewSets, et gestion d'état dans une SPA React.

Ce travail a permis à l'équipe de consolider ses compétences en Python, en conception d'API REST avec Django REST Framework, en développement d'interfaces réactives avec React, et en gestion de projet logiciel collaboratif. Les perspectives d'évolution identifiées sont : authentification par jetons JWT, notifications par e-mail/SMS, génération de bulletins PDF automatisée, et application mobile de pointage par QR Code.